Doc. No.:	WG21/P1217R1
Date:	2019-03-10
Reply-to:	Hans-J. Boehm
Email:	hboehm@google.com
Authors:	Hans-J. Boehm, with input from many others
Audience:	SG1

P1217R1: Out-of-thin-air, revisited, again

Abstract

This is a status update attempting to summarize an external discussion of so-called out-of-thin-air results with `memory_order_relaxed`. It was known that allowing such results often makes it impossible to reason about code using `memory_order_relaxed`, and that our current wording prohibiting them is excessively vague. Many of us believed that this was a stopgap until we determined a better way to word this restriction without invalidating current implementations. It has become much more clear that this cannot happen; current implementations in fact allow results that, unless we redefine the semantics of basic sequential constructs like if-statements, can only be understood as out-of-thin-air results.

There is a well-known and simple solution to this problem that amounts to prohibiting the compiler and hardware from reordering a relaxed load followed by a relaxed store. This remains controversial, mostly because it adds a small amount of overhead to memory_order_relaxed operations on architectures like ARM, and an unknown amount of overhead on at least some GPUs. It also affects the legality of compiler optimizations on `memory_order_relaxed` accesses.

Acknowledgements

This benefited greatly from discussions and debates with many others, including Sarita Adve, Will Deacon, Brian Demsky, Doug Lea, Daniel Lustig, Paul McKenney, and Matt Sinclair. I believe we all agree that there is a problem that should be addressed, but not on a detailed solution.

History

This topic was briefly discussed by SG1 at the Rapperswil meeting. This paper is in part an attempt to respond to the question, raised at that meeting, as to whether we can just live with the results allowed by current implementations.

The paper was discussed in San Diego, without a clear consensus how to proceed.

R1 corrects a mistake in the "data_wrapper" example that was pointed out by Derek Dreyer.

The out-of-thin-air problem

The out-of-thin-air problem has been repeatedly discussed, e.g. in N3710, and http://plv.mpi-sws.org/scfix/paper.pdf.

The fundamental problem is that without an explicit prohibition of such results, the C++ memory model allows `memory_order_relaxed` operations to execute in a way that introduces causal cycles, making it appear that values appeared out of thin air, commonly abbreviated as OOTA. The canonical example is the following, where `x` and `y` are atomic integers initialized to zero:

OOTA Example 1	
Thread 1	Thread 2
`r1 = x.load(memory_order_relaxed);` `y.store(r1, memory_order_relaxed);`	`r1 = y.load(memory_order_relaxed);` `x.store(r1, memory_order_relaxed);`

Without specific requirements to the contrary, the loads are allowed to see the effects of the racing stores. Thus an execution in which the stores write some arbitrary value, say 42, and the loads read those values, is valid. This can be interpreted as an execution in which both loads speculatively return 42, then perform the stores (e.g. to memory only visible to these two threads, where it could be rolled back if we guessed wrong), and then check that the guesses for the loads were correct, which they now are.

Thus with OOTA execution allowed, the above code can set `x` and `y` to a value that is computed nowhere in the code, and can only be justified by a circular argument, or one involving explicit reasoning about speculation.

Similarly, consider the following program in a world with out-of-thin-air results allowed:

OOTA Example 2	
Thread 1	Thread 2

`if (x.load(memory_order_relaxed))`
`  y.store(1,`
`memory_order_relaxed);`
	`if (y.load(memory_order_relaxed))`
	`  x.store(1,`
	`memory_order_relaxed);`

Again both stores can be speculated to occur, and be read by the loads, thus validating the speculation, and causing both variables to be set to 1. This is annoying because the corresponding program written without atomics, is data-race-free, and always does nothing, as expected.

However the worst scenario in this OOTA world is the following. Assume that `x` and `y` are declared as integers, but are only ever assigned 0, 1, or 2, and that functions `f()` and `g()` take one such 0-to-2 argument each. `f()` and `g()`'s precondition effectively includes the restriction that its argument is between 0 and 2.

OOTA Example 3	
Thread 1	Thread 2
`r1 = x.load(memory_order_relaxed))` `f(r1)`	`r2 = y.load(memory_order_relaxed))` `g(r2);`

Again, we have to consider an execution in which each load guesses that the value seen is e.g. 42. This then gets passed to `f()` and `g()`, which are allowed to do anything, since their precondition is not satisfied. "Anything" includes assigning 42 to `y` and `x` respectively, e.g. as a result of an out-of-bounds array reference. This action would again validate the speculation, making it legal.

This last example is particularly disconcerting, since it seems essentially impossible to for the programmer to avoid in real code. It only requires that we perform relaxed loads from two variables in two different threads, and pass the result to some functions that make some assumption about their input. Any significant code performing relaxed loads on pointers is essentially certain to contain a variant of this pattern.

Fortunately nobody believes that any of the above three examples can actually produce these results in real implementations. All common hardware disallows such results by prohibiting a store from speculatively executing before a load on which it "depends". Unfortunately, as we will see below, the hardware notion of "depends" is routine invalidated by many important compiler transformations, and hence this definition does not carry over to compiled programming languages.

Although such results are not believed to occur in practice, the inability to precisely preclude them has serious negative effects. We have no usable and precise rules for reasoning about programs that use `memory_order_relaxed`. This means we have no hope of formally verifying most code that uses `memory_order_relaxed` (or `memory_order_consume`). "Formal verification" includes partial verification to show the absence of certain exploits. Since we don't have a precise semantics to base it on, informal reasoning about `memory_order_relaxed` also becomes harder. Compiler optimization and hardware rules are unclear.

It is also worth noticing that so far we have constrained ourselves to examples in which each thread is essentially limited to two lines of code. The effect on more realistic code is not completely clear. Most of this paper focusses on a new class of examples that we hadn't previously explored.

The status quo

However, we have been unable to provide a meaningful C++-level definition of "out-of-thin-air" results. And thus the standard has not been able to meaningfully prohibit them. The current standard states:

> “Implementations should ensure that no "out-of-thin-air" values are computed that circularly depend on their own computation.”

while offering only examples as to what that means. (See N3786 for a bit more on how we got here. This did not go over well in WG14.)

So far, our hope has been that this problem could be addressed with more ingenuity on our part, by finally developing a proper specification for "out-of-thin-air" results. Here I explain why I no longer believe this is a reasonable expectation.

What changed

Although it was previously known (see e.g. Boehm and Demsky and Bill Pugh et al's much earlier Java memory model litmus tests) that there were borderline cases, which had a lot of similarity to out-of-thin-air results, but could be generated by existing implementations, it wasn't fully clear to us how similar these can get to true out-

of-thin-air results, and the catastrophic damage they can already do. I no longer believe that it is possible to draw a meaningful line between these and "true" out-of-thin-air results. Even if we could draw such a line, it would be meaningless, we would still have no good way to reason about code on the "right" side of the line since, as we show below, that can still produce completely unacceptable results.

Note that we still do not have actual code failures that have been traced to this problem; we do now have small sections of contrived code that can result in the problems discussed here. And we do not have any way to convince ourselves that real code is immune from such problems. The most problematic cases are likely to involve atomics managed by different modules, with the atomic accesses potentially quite far apart. Thus failure rates are likely to be very low, even if the problem does occur in real code.

The central problem continues to be our inability to reason about code using `memory_order_relaxed`.

A new class of out-of-thin-air results

Unfortunately, our initial canonical out-of-thin-air example above can be turned into a slightly more complicated example that can generate the same questionable result. Abstractly, as defined by the standard, the only difference between the two examples is an unexecuted conditional, something that should not make any difference. And I believe it cannot make any difference with anything like the specification we use in the current standard.

In external discussion, such examples have been dubbed "RFUB" ("read from unexecuted branch") instead of OOTA. Our initial example, with additions over OOTA Example 1 highlighted, is:

RFUB Example 1	
Thread 1	Thread 2

`r1 = x.load(memory_order_relaxed);`
`y.store(r1, memory_order_relaxed);`
	`bool assigned_42(false);`
	`r1 = y.load(memory_order_relaxed);`
	`if (r1 != 42) {`
	`  assigned_42 = true;`
	`  r1 = 42;`
	`}`
	`x.store(r1, memory_order_relaxed);`
	`assert_not(assigned_42);`

We argue that entirely conventional optimizations can result in an execution *in which the assertion succeeds*, but `x` and `y` have been assigned 42. In reality, this is achieved with a combination of hardware and compiler transformations.

(We assume that `assert_not` is a separately compiled function, with the implied semantics. In our environment, we get different code if we use the actual `assert` macro.)

The compiler transformations proceed roughly as follows:

1. Notice that the store to `x` must always assign 42. Update it to just store 42.

2. The assignment to `r1` is now dead. Remove it.
3. Replace the remaining conditional, whose body now just assigns to `assigned_42`, with essentially `assigned_42 = (r1 != 42)`.

This gives us essentially:

```
bool assigned_42;
r1 = y.load(memory_order_relaxed);
assigned_42 = (r1 != 42);
x.store(42, memory_order_relaxed);
assert_not(assigned_42);
```

Gcc 7.2 -O2 on Aarch64 (ARMv8) generates (from the source in the table):

```
ldr     w1, [x0]   // w1 = y.load(...)
add     x0, x0, 8  // x0 = &x
mov     w2, 42
str     w2, [x0]   // x = 42
cmp     w1, w2     // w0 = (y != 42)
cset    w0, ne
b       _Z10assert_notb  // call assert_not(w0)
```

Since ARMv8 allows an independent load followed by an independent store to be reordered, the store (`str`) instruction may be executed before any of the other instructions, effectively transforming the code to:

```
x.store(r1, memory_order_relaxed);
bool assigned_42;
r1 = y.load(memory_order_relaxed);
assigned_42 = (r1 != 42);
assert_not(assigned_42);
```

If Thread 1 executes immediately after the store instruction, before any other Thread 2 instructions, the load of `y` will read 42, `assigned_42` will be false, and we will end up with exactly the problematic execution.

Note that there is no real obstacle to this occurring on a strongly ordered architecture like x86 as well; the hardware would not reorder a load with a later store, but there is no rule preventing the compiler from doing so. It is just difficult to construct examples in which that would be a profitable compiler transformation. On the other hand, the hardware may find this to be profitable if e.g. the load misses the cache.

This behavior makes it look as though the then-clause in the condition was partially executed. That is clearly not allowed by the current spec. But, aside from the vague "normative encouragement" to avoid OOTA results, this can be explained as the loads at the beginning of both threads speculatively observing the stores at the end, i.e. as an OOTA result.

Unfortunately, this means that current mainstream implementations are not actually following the "normative encouragement" to avoid OOTA results.

The separate external discussion has considered a number of RFUB examples. They don't seem quite as disastrous as OOTA Example 3 above. But the following section argues that reasoning about programs with RFUB results remains essentially intractable, since it invalidates arguments about properly used sequential code.

Effect on sequential reasoning about code

[This section is not essential for the rest of this discussion.]

Here we focus on the fact that RFUB executions appear to be inconsistent, in that the only way to understand them, other than as a general OOTA result, is as partially executing a conditional clause. This example illustrates that observation more directly by pointing out that expected semantics of correctly called single-threaded code may be violated when combined with concurrent code residing in a separate module. To do so, we look at a larger example, consisting of two different files, which should be thought of as modules written by different authors. This is a thought experiment; I did not compile this code:

data_wrapper.h:

This encapsulates a pointer to a piece of data, along with a binary property of that data. The data is automatically deallocated if it is never handed out to a client:

```
// Concurrent calls on a single object not allowed.
class data_wrapper {
    private:
       Foo *my_data;
       int flags;
       // flag values:
         static const_expr int ESCAPED = 1;
         static const_expr int IS_UNUSUAL = 2;

    public:
       data_wrapper() : flags(0), my_data(new Foo(...)) {}
       ...

       bool is_definitely_normal(Foo * x) {
          // This refers to x, and it hasn't been marked as unusual.
          return my_data == x && (flags & IS_UNUSUAL == 0);
       }

       void set_weird() {
          flags |= IS_UNUSUAL;
       }

       Foo* get_the_data() {
          flags |= ESCAPED;
          return my_data;
       }
```

```
        ~data_wrapper() {
           if (!(flags & ESCAPED)) {
              delete my_data;
           } // otherwise it will go away when its arena is deallocated.
        }

        ...
}
```

Reasonable programmers would expect the delete call in the destructor to only be invoked when `my_data` did not escape, i.e. was never returned by `get_the_data()`. That safety argument is based purely on sequential reasoning; there is no concurrency involved up to this point. It should hold for any caller that does not invoke undefined behavior.

Nonetheless the following concurrent program, which invokes `data_wrapper` member functions from only a single thread, and is thus clearly data-race-free, violates this sequential reasoning, and allows `my_data` to be prematurely deleted.

Note that `IS_UNUSUAL` is introduced only to illustrate how a conditional similar to that in the last example can be introduced in a non-obvious way; it is not crucial to the transformation.

main.cpp:

```
#include "data_wrapper.h"
std::atomic<Foo *> data1;
std::atomic<Foo *> data2;
```

Thread 1 sometimes executes the following. It is the only thread that accesses a `data_wrapper`:

```
{
    data_wrapper dw ....;
    ...
    r1 = data1.load(memory_order_relaxed);
    if (!dw.is_definitely_normal(r1)) {
       r1 = dw.get_the_data();
    }
    data2.store(r1, memory_order_relaxed);
    dw.set_weird();
    ...
}
```

Thread 2 may concurrently copy `data2` to `data1`, also using relaxed operations, as in e.g.

```
r2 = data2.load(memory_order_relaxed);
data1.store(r2, memory_order_relaxed);
```

Transforming `data_wrapper` code

If the `data_wrapper` calls are inlined into Thread 1, we get, for the Thread 1 code between ellipses:

```
r1 = data1.load(memory_order_relaxed);
if (!(dw.my_data == r1  && dw.flags & IS_UNUSUAL == 0)) {
    dw.flags |= ESCAPED;
    r1 = dw.my_data;
}
data2.store(r1, memory_order_relaxed);
dw.flags |= IS_UNUSUAL;
```

The compiler is motivated to temporarily promote the flags field to a register. Since the atomic store is relaxed, this is allowed, and we get:

```
r1 = data1.load(memory_order_relaxed);
rflags = dw.flags;
if (!(dw.my_data == r1 && rflags & IS_UNUSUAL == 0)) {
    rflags |= ESCAPED;
    r1 = dw.my_data;
} // else dw.my_data == r1 && ...
data2.store(r1, memory_order_relaxed);
rflags |= IS_UNUSUAL;
dw.flags = rflags;
```

Applying De Morgan's law to the conditional for clarity, we get:

```
r1 = data1.load(memory_order_relaxed);
rflags = dw.flags;
if (dw.my_data != r1 || rflags & IS_UNUSUAL != 0) {
    rflags |= ESCAPED;
    r1 = dw.my_data;
} // else dw.my_data == r1 && ...
data2.store(r1, memory_order_relaxed);
rflags |= IS_UNUSUAL;
dw.flags = rflags;
```

The compiler can now again conclude that `r1` will always be `dw.my_data` after the conditional, and transform this to (after also observing that the `r1` assignment in the conditional is dead after transforming the store to `data2`):

```
r1 = data1.load(memory_order_relaxed);
rflags = dw.flags;
if (dw.my_data != r1 || rflags & IS_UNUSUAL != 0) {
    rflags |= ESCAPED;
}
```

```
data2.store(dw.my_data, memory_order_relaxed);
rflags |= IS_UNUSUAL;
dw.flags = rflags;
```

which, since `ESCAPED` happens to be 1, can be transformed to

```
r1 = data1.load(memory_order_relaxed);
rflags = dw.flags;
rflags |= (dw.my_data != r1 || rflags & IS_UNUSUAL != 0);
data2.store(dw.my_data, memory_order_relaxed);
rflags |= IS_UNUSUAL;
dw.flags = rflags;
```

This once again allows the atomic store to be advanced before the atomic load, effectively giving us

```
data2.store(dw.my_data, memory_order_relaxed);
// Thread 2 may copy data2 to data1 here.
r1 = data1.load(memory_order_relaxed);
rflags = dw.flags;
rflags |= (dw.my_data r != r1 || rflags & IS_UNUSUAL != 0);
rflags |= IS_UNUSUAL;
dw.flags = rflags;
```

ARM or Power hardware is clearly allowed to advance the stomic store, if the `&&` expression is compiled without branches, which is entirely plausible. That in turn allows Thread 2 to copy `data2` to `data1` between the store and the load, causing the `data1` load to see a value of `dw.my_data`. The `IS_UNUSUAL` bit in `rflags` is zero until we explicitly set it. Thus `rflags` is not altered by the middle assignment to `rflags`, and `ESCAPED` is not added, in spite of the fact that both `data1` and `data2` now contain `dw.my_data`. Thus exiting the scope in which `dw` is declared will delete `dw.my_data`, leaving data1 and data2 as dangling references.

It is completely unclear what the programmer might have done wrong here. The `data_wrapper` implementation is clearly correct. And the author of the client couldn’t really avoid this without carefully inspecting the `data_wrapper` code, violating modularity, and fully understanding potential compiler transformations.

Possible fixes and their cost

The kind of problems we have seen here cannot occur at the assembly language level, or with a naive compiler. Weak hardware memory models like ARM and Power allow the crucial reordering of a load followed by a store, but only if the store does not "depend" on the load. The architectural specification includes a definition of "depend". For our examples, when naively compiled, the final store "depends" on the initial load, so the reordering would be prohibited, preventing the problematic executions.

The core problem is that the architectural definitions of dependence are not preserved by any reasonable compiler. In order to preserve the intent of the hardware rules, and to prevent OOTA behavior, we need to strengthen this notion of dependence to something that can reasonably be preserved by compilation. Many attempts to do so in clever ways have failed. The RFUB examples argue that this is not possible in the context of

the current specification, since the only difference between full OOTA behavior, and behavior allowed by current implementations, is an unexecuted if-branch. And even if we could make this distinction, it wouldn't be useful, since implementations currently allow behavior that we don't know how to reason about.

It seems increasingly clear that the best general solution is to strengthen this notion of "dependence" to simply the "sequenced before" relation, and thus require that a relaxed load followed by a relaxed store cannot be reordered. This clearly has the disadvantage of disallowing current implementations. We previously proposed a variant of this solution in N3710. Any change in this direction may need to be phased in in some way, in order to allow compiler, and possibly hardware, vendors time to adjust.

This is also the solution that was formalized in http://plv.mpi-sws.org/scfix/paper.pdf. The formal treatment is quite simple; we just insist that the "reads from" relation is consistent with "sequenced before". We also expect the standards wording to be quite simple.

The performance of this approach was recently studied in Peizhao Ou and Brian Demsky, "Towards Understanding the Costs of Avoiding Out-of-Thin-Air Results" (OOPSLA'18, should be available by San Diego meeting). The overhead of preventing load; store reordering on ARM or Power is much less than that of enforcing acquire/release ordering. There are some uncertainties about machine memory models in this area, but the expectation is that the required ordering can be enforced by inserting a never-taken branch or the like after `memory_order_relaxed` loads. In many cases, even this will not be needed, because an existing branch, load, or store instruction either already fulfills that role, or can be made to do so. No CPU architecture appears to require an actual expensive fence instruction to enforce load;store ordering.

I am not aware of any cost measurements for enforcing load;store ordering on a GPU. Such measurements would be extremely useful.

Pursuing this path will require some compiler changes around the compilation of `memory_order_relaxed`. And I expect it would eventually result in the introduction of better hardware support than the current "bogus branch" technique.

Participants in the external discussion largely agreed that we need a replacement or replacements for `memory_order_relaxed`. But there is no clear consensus on many more detailed questions:

1. Should we simply strengthen the semantics of `memory_order_relaxed` along the above lines, or leave it as is, and add a stronger version amenable to precise reasoning?
2. It seems likely that even if we replace `memory_order_relaxed` semantics with the stronger semantics, we will want to offer one or more more specialized weaker versions, since some specific usage idioms do not require the stronger treatment. For example, a relaxed atomic increment of a counter often does not require the strengthening, because the result of the implied load is not otherwise used. Should there be one such weaker order, or one for each idiom?
3. If we have such weaker order(s), should they be specified in the same style as now, or as a set of constraints under which they imply sequentially consistent behavior, as in Sinclair, Alsop, and Adve, "Chasing Away RAts: Semantics and Evaluation for Relaxed Atomics on Heterogeneous Systems", ISCA 2017 (a.k.a. DRFrlx).
4. Or should we try to follow the path of DRFrlx, and only have `memory_order_relaxed` replacements that are specified in this way?

My opinion is that, since current implementations effectively do not follow the OOTA guidance in the standard, and since we want to preserve correctness of current code, while actually specifying its behavior precisely, we should change the existing semantics of `memory_order_relaxed` to guarantee load;store ordering.

We should then try to develop weaker memory orders tailored to specific idioms, and specified in a DRFrlx style, to regain any performance lost by the preceding step. I currently do not believe that we will be able to find a single weaker specification that avoids our current OOTA issues. Thus I expect the specifications to be tailored to specific idioms. It currently seems useful to expose these as different `memory_orders` to make it clear what part of the specification the user is relying on. It also seems likely that each of these will impose somewhat different optimization constraints. And many of us are of the opinion that `memory_order_relaxed` is already usable primarily in cases that match one of a small number of reasonably well-known idioms.

I haven't been convinced that the DRFrlx approach by itself is currently viable as a replacement for `memory_order_relaxed`, both due to backwards compatibility issues, and because some of the use cases we see in Android appear to be hard to cover.

The end result here should be a specification that provides performance similar to what we have now, but that is well-defined for all the `memory_orders`, rather than the current hand-waving for `memory_order_relaxed`. It would also address OOTA issues for `memory_order_consume`, but not touch the other `memory_order_consume` issues we have been discussing separately.

Full decisions here will have to wait for some of the missing information, particularly in regard to GPU performance implications. But a preliminary sense of the committee on these issues would be useful.